

Content Fragments & Experience Fragments

A Complete Guide for Adobe Experience Manager (AEM)

1. Overview Comparison

	Content Fragment (CF)	Experience Fragment (XF)
Nature	Pure content (headless)	Content + layout + design
Format	Structured data / text	Full renderable HTML block
Focus	What it says	How it looks
Storage	/content/dam/	/content/experience-fragments/
Editor	CF Editor (form-based)	Page Editor (visual drag & drop)
Output	Raw content / JSON	Rendered HTML with styles
Headless Use	Primary use case	Limited support
Adobe Target	Not applicable	Native integration
Variations	Supported	Supported

2. Content Fragments (CF)

Content Fragments store structured, channel-agnostic content that can be reused across multiple channels — web, mobile, IoT, and more. They are the backbone of headless AEM delivery.

Key Characteristics

- Created using **Content Fragment Models** (define the schema/fields)
- Stored under **/content/dam/** — treated like a DAM asset
- Output is **raw content** — no presentation layer involved
- Ideal for **headless CMS** delivery via AEM's GraphQL API or Content Services
- Supports **variations**: Short, Long, Social versions of the same content
- Multi-field types: text, numbers, dates, references, enumerations, etc.

Structure

Content Fragment Model (schema) → Content Fragment (instance) → Master / Short / Social variations

Technical Details

Node Type	dam:Asset with dam:cfVariations
Model Path	/conf/<project>/settings/dam/cfm/models/
GraphQL API	/graphql/execute.json/...

Component	core/wcm/components/contentfragment
Delivery	GraphQL · Content Services · Sling Model Exporter (JSON)

Use Cases

Product Descriptions	Reused on web, app, kiosk, and email from a single source
Article Content	Delivered to multiple frontends without duplicating data
Structured Data	FAQs, bios, technical specifications, product features
Multi-channel	Content published to web, mobile, smart displays simultaneously

3. Experience Fragments (XF)

Experience Fragments are reusable page sections with full layout, styling, and behavior — built once and used across many pages, channels, or even exported to Adobe Target for personalization.

Key Characteristics

- Stored under `/content/experience-fragments/`
- Composed using the **AEM Page Editor** (visual drag & drop)
- Has **Building Blocks** — sub-fragments reusable within an XF
- Supports multiple **variations per fragment**: web, email, AMP, etc.
- Can be **exported to Adobe Target** for A/B testing and personalization
- Renders as full HTML complete with styles and scripts

Structure

Experience Fragment → Web variation / Email variation / AMP variation → Building Blocks

Technical Details

Node Type	cq:Page with cq:experienceFragments
Path	<code>/content/experience-fragments/</code>
JSON Export	<code>.model.json</code> Sling selector
Target Export	Via Cloud Service configuration
Building Blocks	Reusable sub-components within the same XF

Use Cases

Header / Footer	Site-wide components with nav, logo, and links
Promotional Banners	Reused across multiple pages without copy-paste
Hero Sections / CTAs	Consistent call-to-action blocks with designed layout
Email Templates	Variation for email with appropriate styling
Adobe Target Offers	A/B testing fragments personalized per audience segment

4. Decision Guide — When to Use What

Scenario	Recommended
REST/GraphQL API for React/mobile app	Content Fragment (CF)
Reusable promo banner across 10 pages	Experience Fragment (XF)
Product data for multiple storefronts	Content Fragment (CF)
Site header with nav + logo	Experience Fragment (XF)
Adobe Target A/B test offer	Experience Fragment (XF)
Multi-channel article content	Content Fragment (CF)
Email template with layout	Experience Fragment (XF)
Structured FAQ data	Content Fragment (CF)

Pro Tip: CF inside XF

You can embed a **Content Fragment inside an Experience Fragment** — giving you the flexibility of structured content (CF) with the visual reusability of a designed block (XF). This is a common pattern for editorial content with a consistent layout. The CF handles the *what*, and the XF handles the *how it looks*.